



Department of Information Technology

University of Gujrat

Smart AI-Based Parking Management System

Submitted By:

Mr. Ali Irshad	Reg#22031556-018
Ms. Sabhat Fatima	Reg#22031556-011
Mr. Touseef Ahmed	Reg#22031556-018

Supervised By:

Prof. Sohail Badar

IT- UOG - Project Management Office	Version: 1.0
Final Year Project Proposal Guide	Date: November 2025

DECLARATION

I certify that project title Smart AI-Based Parking Management System is under my supervision with students of BS, Faculty of Computing & Information Technology, University of Gujrat, Pakistan, worked under my supervision.

Prof. Sohail Badar

Supervisor's Name

Department of Information Technology

Faculty of Computing & Information Technology

University of Gujrat, Punjab, Pakistan.

Email: _____@uog.edu.pk

Dated: _____

TABLE OF CONTENTS

FINAL YEAR PROJECT PROPOSAL	4
1.1 INTRODUCTION.....	4
1.2. PROJECT TITLE:.....	4
1.3. PROJECT OVERVIEW STATEMENT:	4
1.4. PROJECT GOALS & OBJECTIVES:	5
1.5. HIGH-LEVEL SYSTEM COMPONENTS:	7
1.6. LIST OF OPTIONAL FUNCTIONAL UNITS:	8
1.7. EXCLUSIONS:	9
1.8. APPLICATION ARCHITECTURE:.....	10
1.9. GANTT CHART:.....	12
1.10. HARDWARE AND SOFTWARE SPECIFICATION:	13
1.11. TOOLS AND TECHNOLOGIES USED WITH REASONING:	15

Final Year Project Proposal

1.1 Introduction

The comprehensive AI-powered parking management system presented in this proposal is intended to address the major issues that Pakistani urban parking facilities face. To automate parking operations, minimize manual labor, and enhance the user experience, the system uses computer vision technologies such as License Plate Detection (LPD) Automated Vehicle Image Taking and Optical Character Recognition (OCR) with a mobile and web-based booking platform. Through automated vehicle recognition and intelligent slot management, the suggested solution helps parking owners to effectively manage their facilities while offering customers a smooth booking, payment, and parking experience.

1.2. Project Title:

Smart AI-Based Parking Management System

1.3. Project Overview statement:

Project Title: Smart AI-Based Parking Management System			
Project Manager: Ali Irshad			
Project Members:			
Name	Registration #	Email Address	Signature
Ali Irshad	22031556-018	ali.irshad3098@gmail.com	
Touseef Ahmed	22031556-042	touseefahmedkiyani85@gmail.com	
Sabhat Fatima	22031556-011	Sabhatfatima662@gmail.com	
Project Goal:			
To create an intelligent, automated parking management system that uses OCR and AI-based license plate detection to streamline parking operations, do away with manual entry/exit procedures, reduce payment fraud, and give users real-time slot availability via mobile and web platforms .			
Objectives:			
Sr.#	Objective		
1	Develop an Android and web-based application for users to search, book, and pay for parking slots online or via cash		
2	Implement AI-powered License Plate Detection (LPD) system for automatic vehicle identification at entry and exit points		
3	Integrate Optical Character Recognition (OCR) for ID card verification with enable/disable functionality		
4	Create comprehensive dashboards for Admin, Parking Owner, and Cashier roles with real-time monitoring capabilities		
5	Develop an automated overstay detection and billing system with grace period management		
6	Implement Machine Learning model to predict peak hours based on historical occupancy data		
7	Design a secure, role-based access control system with Firebase Firestore for real-time data synchronization		

Project Success criteria: 1. LPD-based automated vehicle entry/exit that is successful with a manual backup option. 2. Updates on slot availability in real time on both mobile and web platforms. 3. Precise overstay computation and automated billing production. 4. Safe card and EasyPaisha payment processing (sandbox). 5. Predicting peak hours using machine learning with a minimum accuracy of 75% 6. The system manages concurrent bookings without slot conflicts. 7. Complete role separation between Admin, Owner, Cashier, and User	
Assumptions: 1. Internet connectivity is available at parking locations for real-time updates. 2. Users have access to Android devices or web browsers. 3. Parking owners will collaborate in system deployment and testing. 4. The quality of the cameras at parking locations will be adequate for license plate detection; Risks: 1. Damaged number plates or dim lighting may have an impact on LPD accuracy. 2. The quality of ID card images affects OCR performance. 3. The availability and response time of payment gateway APIs 4. User adoption rate for mobile application Obstacles: 1. Limited availability of real parking sites for testing purposes 2. Dataset collection for ML model training 3. Manual entry fallback process training for cashiers	
Organization Address (if any): University of Gujrat, Gujrat, Punjab, Pakistan	
Type of project: ☐ Research ☒ Development	
Target End users: 1. Vehicle owners seeking parking spaces 2. Parking facility owners and operators 3. Cashiers/counter staff at parking sites 4. System administrators for overall management	
Development Technology: ☒ Object Oriented ☐ Structured	
Platform: ☒ Web based ☒ Distributed ☐ Desktop based ☐ Setup Configurations ☒ Other: <u>Mobile Based / Cross-platform (Flutter)</u>	
Approved By:	
Date:	

1.4. Project Goals & Objectives:

Goals:

This project's main objective is to transform Pakistan's traditional parking management system by implementing an automated, AI-powered system that eliminates manual vehicle registration, lowers payment fraud, reduces human error, and gives users real-time parking availability. By bridging the gap between modern technology and conventional parking operations, the system hopes to improve parking management's effectiveness, transparency, and user-friendliness for both customers and parking owners.

Objectives:

User Application Development: Develop Android and web applications that allow users to register with OTP verification, add vehicle details browse parking websites, reserve slots for one or more days, extend the duration of the reservation, and pay with cash, EasyPaisa, or cards.

AI-Powered Vehicle Recognition: Implement a system for real-time Camera feed License Plate Detection at the entry and exit points to automatically mark the times of entry/exit, identify vehicles, and check the booking status without the need for human intervention.

ID Verification System: Develop an OCR-based ID card scanning module that can be enabled or disabled by parking owners for additional security, capturing and storing ID card images temporarily for verification purposes.

Multi-Role Dashboard System: Create distinct web dashboards for Admin (complete management), Parking Owner (site management, financial gain monitoring, pricing management), and Cashier (vehicle entry control, payment handling, overstay calculation) along with suitable access restrictions.

Intelligent Slot Management: Create an automated slot allocation system with fixed numbered slots (A1, A2, etc.) based on owner-defined capacity, supporting three slot types (Normal, VIP, Disabled) with 10-minute reservation locks and automatic release on payment timeout.

Overstay Detection & Billing: Develop an automatic overstay detection mechanism with 10-minute grace period, generating additional charges based on exceeded time and preventing exit until all payments are cleared.

Peak Hour Prediction: Build a Machine Learning model using historical occupancy data to predict peak hours, enabling informed pricing decisions and better resource allocation for parking owners.

Secure Payment & Refund System: Implement a hybrid payment system supporting online (EasyPaisa, card sandbox) and cash payments with defined refund policy (100% before entry, 50% within first 30 minutes after entry, no refund after 30 minutes).

Real-Time Synchronization: Utilize Firebase Firestore for real-time data synchronization across mobile apps and web dashboards, ensuring instant slot availability updates and booking status changes.

Manual Fallback Mechanism: Provide traditional slip-based entry system for cashiers when AI components (LPD/OCR) fail, ensuring system reliability and continuity of operations

1.5. High-level system components:

The Smart AI-Based Parking Management System consists of five primary functional components that work together to deliver a complete parking automation solution:

1. User Module (Mobile & Web Application)

This component serves as the primary interface for vehicle owners seeking parking services. Users can create accounts with OTP verification, manage personal information, register multiple vehicles with complete details (model, color, plate number, vehicle photo), search available parking sites, view real-time slot availability by type (Normal, VIP, Disabled), book slots for single or multiple days, extend existing bookings, make payments through EasyPaisa or card (sandbox), pay cash at exit, view booking history with status (paid/unpaid), receive QR code-based payment slips, and cancel bookings with automatic refund processing based on defined policies.

2. Admin Module (Web Dashboard)

The Admin module provides centralized system control and monitoring capabilities. Admin users can add, modify, or remove parking sites from the system, approve or block parking owner registrations, manage user accounts and resolve disputes, view consolidated payment records across all sites, access comprehensive system logs for entry/exit activities, monitor AI model performance and accuracy, define and override global pricing rules, export detailed reports in PDF/Excel formats for analysis, and configure system-wide settings including grace periods and overstay charges.

3. Parking Owner Module (Web Dashboard)

This component empowers parking facility owners with complete site management tools. Owners can register their parking sites by entering capacity and location details, define slot numbers automatically generated based on capacity (A1-A50), categorize slots by type (Normal, VIP, Disabled), set base pricing for each slot type, adjust pricing during peak hours identified by ML predictions, view total, available, and booked slots in real-time, accept or decline booking requests if manual approval is enabled, monitor daily, weekly, and monthly revenue with graphical representations, enable or disable the optional ID card verification feature, manage cashier accounts assigned to their sites, and access comprehensive booking and payment reports.

4. Cashier/Counter Module (Web Dashboard)

The Cashier module facilitates on-site parking operations and manual interventions. Cashiers can monitor live parking area through camera feed integration, view incoming vehicle arrivals with LPD-detected plate numbers, verify booking status automatically displayed by the system, process both online and cash payments at entry or exit, manually enter vehicle details using traditional slip system when AI fails, calculate overstay charges automatically after 10-minute grace period, process exit requests only after payment clearance, handle walk-in customers without prior bookings through instant slot assignment, update slot status manually when required, and generate payment receipts for cash transactions.

5. AI & Machine Learning Module (Backend Services)

This component forms the technological backbone of the system's automation capabilities. The module performs real-time License Plate Detection using camera feeds at entry/exit gates, applies OCR technology to extract plate numbers from captured images, optionally processes ID card images when verification is enabled, automatically logs entry and exit timestamps in the database, matches detected plate numbers with existing bookings, checks slot availability before confirming bookings, auto-assigns slots for walk-in customers, predicts peak hours using Machine Learning model trained on historical occupancy data, provides prediction inputs to owners for dynamic pricing decisions, processes sample videos for testing purposes during development, and maintains manual fallback protocols when camera or OCR systems fail.

1.6. List of optional functional units:

1. Parking Navigation & Maps Integration

Integrate Google Maps API to show parking site locations, provide turn-by-turn navigation, and display distance from user's current location. This feature enhances user experience but requires additional API costs and location permissions.

2. QR Code Scanner at Entry/Exit

Allow cashiers to scan QR codes from user payment slips for quick verification instead of manual plate number entry. Implementation requires QR code generation in mobile app and scanner integration in cashier dashboard.

3. Loyalty & Discount System

Implement points-based loyalty program for frequent users, referral bonuses, and promotional discount codes. This feature requires additional database tables and business

logic but can improve user retention.

4. Automated Customer Support Chatbot

Implement AI-powered chatbot for handling common user queries, booking assistance, and issue reporting. This feature enhances user support but requires natural language processing implementation.

1.7. Exclusions:

The following features and functionalities will **NOT** be included in this project scope due to technical constraints, time limitations, and resource availability:

1. Cloud Deployment & IoT Integration

The system will **NOT** be deployed on cloud platforms (AWS, Azure, Google Cloud) or involve any IoT hardware devices such as smart barriers, sensors, or connected cameras. All testing will be conducted on local machines, and camera functionality will be simulated using sample videos or images rather than live IoT camera feeds.

2. Bike/Motorcycle Parking Management

The system will **exclusively** handle car parking and will not support bike or motorcycle parking slots. Only three vehicle types are supported: Normal Cars, VIP Cars, and Disabled Cars.

3. Advanced ML Features

Machine Learning will be **limited** to peak hour prediction only. The system will **NOT** implement ML-based dynamic pricing, occupancy forecasting beyond peak hours, or predictive maintenance. Pricing will remain rule-based and controlled manually by parking owners.

4. Real-Time Live Camera Streaming

While the system uses camera images for LPD and OCR, it will **NOT** provide continuous live video streaming to dashboards. Cashiers will see captured snapshots only, not real-time video feeds.

5. Automated Barrier Control

The system will **NOT** control physical entry/exit barriers automatically. Barrier operation will remain manual, triggered by cashiers after system verification, or parking sites may

IT- UOG - Project Management Office	Version: 1.0
Final Year Project Proposal Guide	Date: November 2025

continue using existing barrier systems independently.

6. Multi-Currency & International Payments

Payments are **limited** to Pakistani Rupees (PKR) only. International payment gateways, credit cards beyond sandbox testing, and multi-currency support are excluded.

7. Vehicle Damage Recording & Insurance Claims

The system does **NOT** handle vehicle damage documentation, insurance claim processing, or liability management for damages occurring within parking premises.

8. Multi-City or Franchise Management

The system does **NOT** include franchise management features, multi-city administrative controls, or centralized operations for parking chains across different cities.

9. Biometric Authentication

Fingerprint, facial recognition, or other biometric authentication methods for users are **NOT** included. Authentication relies on OTP-based phone verification only.

1.8. Application Architecture:

The Smart AI-Based Parking Management System follows a **distributed three-tier architecture** with clear separation of concerns between presentation layer, business logic layer, and data layer. This architecture ensures scalability, maintainability, and real-time data synchronization across multiple platforms.

Smart Parking System Architecture

Three-Tier Distributed Architecture

1 Presentation Layer

Mobile App

Flutter Framework

User booking, payments & real-time updates

Web Dashboard

React.js Framework

Admin, Owner & Cashier portals



2 Application Layer

REST API Gateway

Django REST

Business Logic

User, Booking, Payment Management

AI Services

YOLOv8 LPD & EasyOCR

ML Services

Peak Hour Prediction



3 Data Layer

Firebase Firestore

NoSQL Database

Users & Vehicles

Sites & Slots

Bookings

Payments & Logs

● Real-time Sync ● Offline Support ● Transaction Safety

4 Security Layer (Cross-Cutting)

JWT & Firebase Auth

Role-based Access

Data Encryption

Audit Logging

1.9. Gantt chart:



1.10. Hardware and Software Specification:

Hardware Requirements:

Development Environment:

- **Processor:** Intel Core i5 (8th Gen) or higher / AMD Ryzen 5 or equivalent
- **RAM:** Minimum 8GB (16GB recommended for AI model training)
- **Storage:** 256GB SSD minimum (512GB recommended)
- **Display:** 1920x1080 resolution minimum
- **Network:** Stable internet connection for Firebase and API testing

Testing Equipment:

- Android smartphone (minimum Android 8.0) for mobile app testing
- Camera (phone camera or webcam) for capturing sample vehicle images during testing
- Multiple devices for concurrent user testing scenarios

Deployment Server (Local Machine for Demo):

- **Processor:** Intel Core i7 or equivalent
- **RAM:** 16GB minimum
- **Storage:** 500GB SSD
- **Operating System:** Ubuntu 20.04 LTS or Windows 10/11 Pro
- **Network:** Local network setup for client-server communication during demo

Software Requirements:

Development Tools:

- **IDE:** Visual Studio Code, PyCharm (for Python AI modules), Android Studio (for Flutter)
- **Version Control:** Git and GitHub for source code management
- **API Testing:** Postman for REST API endpoint testing
- **Database Management:** Firebase Console for Firestore management

Backend Development:

- **Programming Language:** Python 3.9+
- **Framework:** Django 4.2, Django REST Framework 3.14
- **Database:** Firebase Firestore (NoSQL cloud database)
- **Authentication:** Firebase Authentication, JWT tokens

AI & Machine Learning:

- **Computer Vision:** OpenCV 4.8+
- **License Plate Detection:** YOLOv8 (Ultralytics)
- **OCR:** EasyOCR or Tesseract OCR
- **Machine Learning:** Scikit-learn 1.3+, NumPy, Pandas
- **Image Processing:** PIL (Pillow)

Mobile Application Development:

- **Framework:** Flutter 3.16+
- **Programming Language:** Dart
- **State Management:** Provider or Riverpod
- **Local Storage:** Shared Preferences
- **HTTP Client:** Dio package for API calls

Web Dashboard Development:

- **Framework:** React.js 18+
- **Programming Language:** JavaScript (ES6+)
- **UI Library:** Material-UI or Tailwind CSS
- **State Management:** Redux or Context API
- **HTTP Client:** Axios

Payment Integration:

- **EasyPaiza:** EasyPaiza Merchant API (sandbox for testing)
- **Card Payments:** Stripe sandbox or similar test gateway

Operating Systems:

- **Development:** Windows 10/11, macOS, or Ubuntu 20.04+
- **Deployment:** Ubuntu 20.04 LTS (local server for demo)
- **Mobile:** Android 8.0 (Oreo) and above

Browser Compatibility (for Web Dashboards):

- Google Chrome 100+
- Mozilla Firefox 100+
- Microsoft Edge 100+

Additional Software:

- **Video Processing:** FFmpeg (for sample video handling during AI testing)

- **Image Editing:** Any tool for preparing test images
- **Documentation:** Microsoft Word, LaTeX for report preparation
- **Diagram Tools:** Draw.io, Lucidchart for system diagrams

1.11. Tools and technologies used with reasoning:

1. Flutter (Mobile Application Framework)

Flutter is used because it allows building both Android and web apps from a single codebase. It runs smoothly and updates real-time slot availability instantly. Its ready-made widgets save development time. With direct Firebase integration and hot reload, changes appear immediately, which is very helpful for an FYP project.

2. Django REST Framework (Backend API)

Django REST Framework is chosen for the backend because it is secure and reliable. It supports login and authentication using JWT tokens. It handles data efficiently, allowing APIs to be built with minimal code. Being Python-based, it easily integrates with AI and ML modules.

3. Firebase Firestore (NoSQL Database)

Firestore is used as the database because it updates data in real-time. When a slot becomes free or occupied, the change appears instantly on all devices. It works offline too and requires no separate server management. Its free tier is sufficient for FYP testing and demonstration.

4. YOLOv8 (License Plate Detection)

YOLOv8 is used for detecting number plates because it is fast and accurate. It works in real-time and can be trained with a small dataset. It can be customized for Pakistani vehicle plates, making it ideal for parking entry and exit scenarios.

5. EasyOCR (Text Recognition)

EasyOCR extracts text from images like license plates and ID cards. It supports both English and Urdu, which is useful for Pakistan. It is easy to use, doesn't require extensive training, and works well even with low-quality images.

6. Scikit-learn (Machine Learning Library)

Scikit-learn is used to predict peak hours. It offers simple ML algorithms that are easy to understand and implement. It doesn't require high-end hardware and provides good results suitable for an FYP project.

7. React.js (Web Dashboard)

React.js is used for building the web dashboard. It creates fast and responsive websites. Building separate dashboards for Admin and Owner is easy. When connected with Firebase, it shows real-time data updates.

8. EasyPaisa API (Payment System)

EasyPaisa is used for payments because it is widely used in Pakistan. Payments are confirmed instantly, allowing slots to be booked immediately. Its sandbox environment lets us test without real money, reducing risk during development.

9. Stripe (Card Payment Testing)

Stripe is included for card payment testing. It allows testing with fake cards and simulates different payment scenarios. It helps understand international-standard payments and is easy to implement.

10. OpenCV (Image Processing)

OpenCV is used to preprocess images, such as improving clarity or finding edges. It enhances YOLO and OCR accuracy. It is free, widely used, and very reliable for image processing tasks.

11. Git & GitHub (Version Control)

Git and GitHub are used to save and manage code. They allow rolling back to previous versions if mistakes happen. They also support teamwork and supervisor reviews, making project management easier.

12. Postman (API Testing)

Postman is used to test APIs. It checks backend functionality before frontend integration, helping find errors early and making the system more reliable.

13. Visual Studio Code (IDE)

VS Code is used as the coding environment. It is fast, user-friendly, and supports tools for Flutter, Python, and React, simplifying the development process.